

1)

1. TABLES

```
CREATE TABLE Customer (  
  Custid INT PRIMARY KEY,  
  Custname VARCHAR(50),  
  Age INT,  
  Phone VARCHAR(15)  
);
```

```
CREATE TABLE Loan (  
  Loanid INT PRIMARY KEY,  
  Amount INT,  
  Custid INT,  
  EMI INT,  
  FOREIGN KEY (Custid) REFERENCES Customer(Custid)  
);
```

2. INSERT DATA

```
INSERT INTO Customer VALUES  
(1,'Arun',30,'9876543210'),  
(2,'Bala',28,'9123456780'),  
(3,'Cathy',35,'9988776655'),  
(4,'David',40,'9090909090');
```

```
INSERT INTO Loan VALUES  
(101,60000,1,2000),  
(102,40000,2,1500),  
(103,80000,1,3000),  
(104,20000,3,1000);
```

3. QUERIES

(a) Loan > 50000 Customers

```
SELECT Custname  
FROM Customer C, Loan L  
WHERE C.Custid = L.Custid  
AND Amount > 50000;
```

(b) Customers with NO loan

```
SELECT Custid  
FROM Customer  
WHERE Custid NOT IN (SELECT Custid FROM Loan);
```

(c) Total Loans

```
SELECT COUNT(*) FROM Loan;
```

4. PROCEDURE

```
DELIMITER $$
```

```
CREATE PROCEDURE GetLoan(IN id INT)  
BEGIN  
SELECT Amount, Custid  
FROM Loan  
WHERE Loanid = id;  
END $$
```

```
DELIMITER ;
```

CALL PROCEDURE

```
CALL GetLoan(101);
```

2)

1. TABLES

```
CREATE TABLE Dept (  
  Deptno INT PRIMARY KEY,  
  Dname VARCHAR(50),  
  Loc VARCHAR(50),  
  DeptmanagerId INT  
);
```

```
CREATE TABLE Employee (  
  EmpId INT PRIMARY KEY,  
  Empname VARCHAR(50),  
  Sal INT,  
  Deptno INT,  
  FOREIGN KEY (Deptno) REFERENCES Dept(Deptno)  
);
```

2. INSERT DATA

```
INSERT INTO Dept VALUES  
(10,'HR','Chennai',1),  
(20,'IT','Bangalore',2),  
(30,'Sales','Delhi',3);
```

```
INSERT INTO Employee VALUES  
(1,'Arun',50000,10),  
(2,'Bala',60000,20),  
(3,'Cathy',55000,30),  
(4,'David',40000,10),  
(5,'Esha',70000,20);
```

3. QUERIES

(a) Count & Average Salary

```
SELECT Deptno, COUNT(*), AVG(Sal)  
FROM Employee  
GROUP BY Deptno;
```

(b) Employee + Department + Salary

```
SELECT Empname, Dname, Sal  
FROM Employee E, Dept D  
WHERE E.Deptno = D.Deptno;
```

(c) Employee & Manager Name

```
SELECT E.Empname, M.Empname  
FROM Employee E, Dept D, Employee M  
WHERE E.Deptno = D.Deptno  
AND D.DeptmanagerId = M.Empld;
```

4. FUNCTION

```
DELIMITER $$
```

```
CREATE FUNCTION GetSalary(id INT)  
RETURNS INT  
BEGIN  
DECLARE s INT DEFAULT 0;
```

```
SELECT Sal INTO s  
FROM Employee  
WHERE Empld = id;
```

```
RETURN s;  
END $$
```

```
DELIMITER ;
```

CALL FUNCTION

```
SELECT GetSalary(1);
```

3)

1. TABLES

```
CREATE TABLE Booth (  
  Boothid INT PRIMARY KEY,  
  Location VARCHAR(50),  
  BIncharge VARCHAR(50)  
);
```

```
CREATE TABLE Voter (  
  VoterId INT PRIMARY KEY,  
  Votername VARCHAR(50),  
  Gender VARCHAR(10),  
  Boothid INT,  
  Checkvote INT, -- 1 = voted, 0 = not voted  
  FOREIGN KEY (Boothid) REFERENCES Booth(Boothid)  
);
```

2. QUERIES

(a) Count of voters in each booth

```
SELECT Boothid, COUNT(*) AS Total_Voters  
FROM Voter  
GROUP BY Boothid;
```

(b) Total voters who voted

```
SELECT COUNT(*) AS Total_Voted  
FROM Voter  
WHERE Checkvote = 1;
```

(c) Booth details with voted count

```
SELECT B.Boothid, B.Location, COUNT(V.VoterId) AS Voted_Count  
FROM Booth B, Voter V  
WHERE B.Boothid = V.Boothid  
AND V.Checkvote = 1  
GROUP BY B.Boothid, B.Location;
```

3. FUNCTION (Poll Percentage)

DELIMITER \$\$

```
CREATE FUNCTION PollPercent(bid INT)
RETURNS DECIMAL(5,2)
BEGIN
    RETURN (
        SELECT (SUM(Checkvote = 1) * 100.0) / COUNT(*)
        FROM Voter
        WHERE Boothid = bid
    );
END $$
```

DELIMITER ;

CALL FUNCTION

```
SELECT PollPercent(1);
```

4)

1. TABLES

```
CREATE TABLE Book (  
  Bookid INT PRIMARY KEY,  
  Book_name VARCHAR(50),  
  Publication VARCHAR(50),  
  Price INT  
);
```

```
CREATE TABLE User (  
  Userid INT PRIMARY KEY,  
  Name VARCHAR(50),  
  Bookid INT,  
  FOREIGN KEY (Bookid) REFERENCES Book(Bookid)  
);
```

2. INSERT DATA

BOOK TABLE

```
INSERT INTO Book VALUES  
(101,'DBMS','Wiley',600),  
(102,'OS','Pearson',500),  
(103,'AI','Wiley',800),  
(104,'CN','McGrawHill',700);
```

USER TABLE

```
INSERT INTO User VALUES  
(1,'Arun',101),  
(2,'Bala',103),  
(3,'Cathy',102),  
(4,'David',104);
```

3. QUERIES

(a) User who accessed costliest book

```
SELECT Name  
FROM User U, Book B  
WHERE U.Bookid = B.Bookid  
AND Price = (SELECT MAX(Price) FROM Book);
```

(b) User id + count of books

```
SELECT Userid, COUNT(*)  
FROM User  
GROUP BY Userid;
```

(c) Books published by Wiley

```
SELECT *  
FROM Book  
WHERE Publication='Wiley';
```

4. FUNCTION

```
DELIMITER $$
```

```
CREATE FUNCTION GetPrice(bid INT)  
RETURNS INT  
BEGIN  
RETURN (SELECT Price FROM Book WHERE Bookid=bid);  
END $$
```

```
DELIMITER ;
```

CALL FUNCTION

```
SELECT GetPrice(103);
```


5)

1. TABLES

```
CREATE TABLE Customer (  
Custid INT PRIMARY KEY,  
Custname VARCHAR(50),  
Addr VARCHAR(100),  
phno VARCHAR(15),  
panno VARCHAR(15)  
);
```

```
CREATE TABLE Loan (  
Loanid INT PRIMARY KEY,  
Amount INT,  
Interest INT,  
Custid INT,  
FOREIGN KEY (Custid) REFERENCES Customer(Custid)  
);
```

```
CREATE TABLE Account (  
Acctno INT PRIMARY KEY,  
Accbal INT,  
Custid INT,  
FOREIGN KEY (Custid) REFERENCES Customer(Custid)  
);
```

2. INSERT DATA

```
INSERT INTO Customer VALUES  
(1,'ARUN','Chennai','9876543210','PAN1'),  
(2,'BALA','Chennai','9123456780','PAN2'),  
(3,'CATHY','Madurai','9988776655','PAN3'),  
(4,'DAVID','Delhi','9090909090','PAN4');
```

```
INSERT INTO Loan VALUES  
(101,50000,5,1),  
(102,80000,6,2),  
(103,60000,4,3),  
(104,40000,3,4);
```

```
INSERT INTO Account VALUES  
(201,30000,1),  
(202,90000,2),  
(203,50000,3),  
(204,20000,4);
```

3. QUERIES

(a) Account balance of ARUN

```
SELECT Accbal
FROM Account
WHERE Custid = (SELECT Custid FROM Customer WHERE Custname='ARUN');
```

(b) Update interest (1%)

```
UPDATE Loan
SET Interest = 1
WHERE Custid IN (
SELECT Custid FROM Account
WHERE Accbal > 0.5 * Amount
);
```

(c) View (Account + Loan)

```
CREATE VIEW Cust_View AS
SELECT Customer.Custname, Account.Accbal, Loan.Amount
FROM Customer, Account, Loan
WHERE Customer.Custid = Account.Custid
AND Customer.Custid = Loan.Custid;
```

Display View

```
SELECT * FROM Cust_View;
```

4. TRIGGER

Step 1: Log Table

```
CREATE TABLE Account_Log (
Acctno INT,
Accbal INT
);
```

Step 2: Trigger

DELIMITER \$\$

```
CREATE TRIGGER after_account_update
AFTER UPDATE ON Account
FOR EACH ROW
BEGIN
INSERT INTO Account_Log VALUES (NEW.Acctno, NEW.Accbal);
END $$
```

DELIMITER ;

STEP-BY-STEP OUTPUT PROCESS For trigger(*Not part of the program)

1. First check original data

```
SELECT * FROM Account;
```

2. Perform UPDATE (this activates trigger)

```
UPDATE Account
SET Accbal = 75000
WHERE Acctno = 201;
```

3. NOW check output (Log table)

```
SELECT * FROM Account_Log;
```

EXPECTED OUTPUT

If trigger works, you will see:

Acctno	Accbal
201	75000

6) 1. TABLES

```
CREATE TABLE Customer (  
  Custid INT PRIMARY KEY,  
  Custname VARCHAR(50),  
  phno VARCHAR(15),  
  pan VARCHAR(20),  
  DOB DATE  
);
```

```
CREATE TABLE HomeLoan (  
  HLoanid INT PRIMARY KEY,  
  Amount DECIMAL(10,2),  
  Custid INT,  
  FOREIGN KEY (Custid) REFERENCES Customer(Custid)  
);
```

```
CREATE TABLE VehicleLoan (  
  VLoanid INT PRIMARY KEY,  
  Amount DECIMAL(10,2),  
  Custid INT,  
  FOREIGN KEY (Custid) REFERENCES Customer(Custid)  
);
```

2. INSERT DATA

```
INSERT INTO Customer VALUES  
(1,'Arun','9876543210','ABCDE1234F','1998-05-10'),  
(2,'Bala','9876543211','PQRSX5678K','1997-08-15'),  
(3,'Charan','9876543212','LMNOP4321Z','1999-02-20'),  
(4,'Divya','9876543213','ASDFG9876H','2000-11-25');
```

```
INSERT INTO HomeLoan VALUES  
(101,500000,1),  
(102,700000,2);
```

```
INSERT INTO VehicleLoan VALUES  
(201,200000,1),  
(202,300000,3);
```

3. QUERIES

(a) Custid of customers who have BOTH HomeLoan and VehicleLoan

```
SELECT Custid FROM HomeLoan  
INTERSECT  
SELECT Custid FROM VehicleLoan;
```

(b) Custid of customers who do NOT have any loan

```
SELECT Custid
FROM Customer
WHERE Custid NOT IN (
    SELECT Custid FROM HomeLoan
    UNION
    SELECT Custid FROM VehicleLoan
);
```

(c) VIEW (Customer id, name, total loan amount)

```
CREATE VIEW LoanView AS
SELECT c.Custid,
       c.Custname,
       h.Amount + v.Amount AS TotalLoan
FROM Customer c
JOIN HomeLoan h ON c.Custid = h.Custid
JOIN VehicleLoan v ON c.Custid = v.Custid;
```

View Output

```
SELECT * FROM LoanView;
```

(d) TRIGGER (Display HomeLoan details after insert)

```
DELIMITER $$
```

```
CREATE TRIGGER ShowHomeLoan
AFTER INSERT ON HomeLoan
FOR EACH ROW
BEGIN
    SELECT NEW.HLoanid AS LoanID,
           NEW.Amount AS LoanAmount,
           NEW.Custid AS CustomerID;
END $$
```

```
DELIMITER ;
```

7) 1. TABLES

```
CREATE TABLE Product (  
  Prodid INT PRIMARY KEY,  
  Prodesc VARCHAR(50),  
  Price DECIMAL(10,2),  
  Stock INT  
);
```

```
CREATE TABLE Sales (  
  Salesid INT PRIMARY KEY,  
  Prodid INT,  
  Qty INT,  
  FOREIGN KEY (Prodid) REFERENCES Product(Prodid)  
);
```

2. INSERT

```
INSERT INTO Product VALUES  
(1,'Laptop',50000,20),  
(2,'Mobile',20000,30),  
(3,'Mouse',500,100);
```

```
INSERT INTO Sales VALUES  
(101,1,5),  
(102,2,10),  
(103,1,2),  
(104,3,15);
```

3. QUERIES

(a) Add REORDER column with value 50

```
ALTER TABLE Product  
ADD Reorder INT DEFAULT 50;
```

(b) Product ID in descending order of quantity ordered

```
SELECT Prodid, SUM(Qty) AS TotalQty  
FROM Sales  
GROUP BY Prodid  
ORDER BY TotalQty DESC;
```

c. FUNCTION

```
DELIMITER $$
```

```
CREATE PROCEDURE Fibonacci(IN n INT)
```

```

BEGIN
  DECLARE a INT DEFAULT 0;
  DECLARE b INT DEFAULT 1;
  DECLARE c INT;
  DECLARE i INT DEFAULT 1;

  SELECT a AS Fibonacci;

  WHILE i < n DO
    SET c = a + b;
    SELECT c AS Fibonacci;
    SET a = b;
    SET b = c;
    SET i = i + 1;
  END WHILE;

END $$

DELIMITER ;

```

Call Procedure

```
CALL Fibonacci(10);
```

d. LOGIN FORM (MySQL IMPLEMENTATION)

Create table (user storage)

```

CREATE TABLE Users (
  Username VARCHAR(50) PRIMARY KEY,
  Password VARCHAR(50)
);

```

Insert users

```

INSERT INTO Users VALUES
('admin','admin123'),
('user1','pass123');

```

Login validation

```

SELECT *
FROM Users
WHERE Username='admin'
AND Password='admin123';

```

8) 1. TABLES

```
CREATE TABLE Product (  
    Prodid INT PRIMARY KEY,  
    Prodesc VARCHAR(50),  
    Price DECIMAL(10,2),  
    Stock INT  
);
```

```
CREATE TABLE Sales (  
    Salesid INT PRIMARY KEY,  
    Prodid INT,  
    Qty INT,  
    FOREIGN KEY (Prodid) REFERENCES Product(Prodid)  
);
```

2. INSERT

```
INSERT INTO Product VALUES  
(1,'Laptop',50000,20),  
(2,'Mobile',20000,30),  
(3,'Mouse',500,100);
```

```
INSERT INTO Sales VALUES  
(101,1,5),  
(102,2,10),  
(103,1,2),  
(104,3,15);
```

3. QUERIES

(a) Add REORDER column with value 50

```
ALTER TABLE Product  
ADD Reorder INT DEFAULT 50;
```

(b) Sales Report

```
SELECT  
    S.Salesid,  
    P.Prodid,  
    P.Prodesc,  
    S.Qty,  
    S.Qty * P.Price AS TotalAmount  
FROM Sales S  
JOIN Product P  
ON S.Prodid = P.Prodid;
```


(c) Product ID in descending order of quantity ordered

```
SELECT
  Prodid,
  SUM(Qty) AS TotalQty
FROM Sales
GROUP BY Prodid
ORDER BY TotalQty DESC;
```

(d) USER LOGIN

```
CREATE TABLE Users (
  Username VARCHAR(50) PRIMARY KEY,
  Password VARCHAR(50)
);
```

Login Check:

```
SELECT 1
FROM Users
WHERE Username='admin'
AND Password='admin123';
```

STOCK AVAILABILITY CHECK

```
SELECT
  Prodid,
  Prodesc,
  Stock,
  CASE
    WHEN Stock > 0 THEN 'AVAILABLE'
    ELSE 'OUT OF STOCK'
  END AS Status
FROM Product;
```

9) 1. TABLES

```
CREATE TABLE Member (  
  MemberCode INT PRIMARY KEY,  
  MemberName VARCHAR(50),  
  PhoneNo VARCHAR(15)  
);
```

```
CREATE TABLE Book (  
  BookCode INT PRIMARY KEY,  
  CategoryCode INT,  
  BookName VARCHAR(50),  
  Cost DECIMAL(10,2)  
);
```

```
CREATE TABLE Issue (  
  MemberCode INT,  
  BookCode INT,  
  IssueDate DATE,  
  ReturnDate DATE,  
  FOREIGN KEY (MemberCode) REFERENCES Member(MemberCode),  
  FOREIGN KEY (BookCode) REFERENCES Book(BookCode)  
);
```

2. INSERT DATA

```
INSERT INTO Member VALUES  
(1,'Arun',9876543210),  
(2,'Bala',9123456780),  
(3,'Charan',9988776655);
```

```
INSERT INTO Book VALUES  
(101,1,'DBMS',500),  
(102,2,'Java',300),  
(103,1,'SQL',450);
```

```
INSERT INTO Issue VALUES  
(1,101,'2007-10-08','2007-10-15'),  
(2,102,'2007-10-08','2007-10-16'),  
(3,103,'2007-10-09','2007-10-20');
```

3. QUERIES

(a) Members who took book on 8-10-2007

```
SELECT M.MemberName  
FROM Member M, Issue I
```

```
WHERE M.MemberCode = I.MemberCode  
AND I.IssueDate = '2007-10-08';
```

(b) Member codes who took books worth more than 400

```
SELECT I.MemberCode  
FROM Issue I, Book B  
WHERE I.BookCode = B.BookCode  
AND B.Cost > 400;
```

(c) CURSOR

```
DELIMITER $$
```

```
CREATE PROCEDURE ShowLibrary()  
BEGIN  
    DECLARE done INT DEFAULT 0;  
    DECLARE mcode INT;  
    DECLARE mname VARCHAR(50);  
  
    DECLARE cur CURSOR FOR  
        SELECT MemberCode, MemberName FROM Member;  
  
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;  
  
    OPEN cur;  
  
    LOOP_FETCH: LOOP  
        FETCH cur INTO mcode, mname;  
        IF done THEN  
            LEAVE LOOP_FETCH;  
        END IF;  
  
        SELECT mcode, mname;  
    END LOOP;  
  
    CLOSE cur;  
END $$  
  
DELIMITER ;
```

Call Procedure:

```
CALL ShowLibrary();
```

(d) REGISTRATION FORM

```
CREATE TABLE LibraryUser (  
  MemberCode INT PRIMARY KEY,  
  MemberName VARCHAR(50),  
  PhoneNo VARCHAR(15)  
);
```

```
INSERT INTO LibraryUser VALUES  
(1,'Arun',9876543210),  
(2,'Bala',9123456780);
```

Display users

```
SELECT * FROM LibraryUser;
```

10) 1. TABLES

```
CREATE TABLE Marks (  
    Regno INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Dept VARCHAR(20),  
    Subj1 INT,  
    Subj2 INT,  
    Subj3 INT  
);
```

2. INSERT

```
(1,'Arun','CSE',80,85,90),  
(2,'Bala','CSE',70,75,80),  
(3,'Charan','IT',90,88,95);
```

3. QUERIES

(a) Add Total column and update it

```
ALTER TABLE Marks ADD Total INT;
```

```
UPDATE Marks  
SET Total = Subj1 + Subj2 + Subj3;
```

(b) Find second maximum total

```
SELECT MAX(Total) AS Second_Max_Total  
FROM Marks  
WHERE Total < (SELECT MAX(Total) FROM Marks);
```

(c) Student with maximum total

```
SELECT Name  
FROM Marks  
WHERE Total = (SELECT MAX(Total) FROM Marks);
```

(d) PL/SQL

```
DELIMITER $$
```

```
CREATE PROCEDURE StudentReport()  
BEGIN  
    DECLARE done INT DEFAULT 0;
```

```

DECLARE rno INT;
DECLARE sname VARCHAR(50);
DECLARE dpt VARCHAR(20);
DECLARE t INT;

DECLARE cur CURSOR FOR
    SELECT Regno, Name, Dept, Total FROM Marks;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

OPEN cur;

read_loop: LOOP
    FETCH cur INTO rno, sname, dpt, t;

    IF done = 1 THEN
        LEAVE read_loop;
    END IF;

    SELECT rno AS Regno, sname AS Name, dpt AS Dept, t AS Total;
END LOOP;

CLOSE cur;
END $$

DELIMITER ;

```

Call Procedure:

```
CALL StudentReport();
```

11) 1. TABLES

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    Dept_Name VARCHAR(50),  
    HoD VARCHAR(50)  
);
```

```
CREATE TABLE Student (  
    Regno INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Gender VARCHAR(10),  
    DeptID INT,  
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)  
);
```

2. INSERT

```
INSERT INTO Department VALUES  
(1,'CSE','Dr. Kumar'),  
(2,'IT','Dr. Suresh'),  
(3,'ECE','Dr. Ram');
```

```
INSERT INTO Student VALUES  
(101,'Arun','M',1),  
(102,'Bala','M',1),  
(103,'Charan','M',2),  
(104,'Divya','F',3);
```

3. QUERIES

(a) View for CSE Department students

```
CREATE VIEW CSE_Dept AS  
SELECT *  
FROM Student  
WHERE DeptID = 1;
```

(b) Department-wise student count (descending order)

```
SELECT DeptID, COUNT(*) AS Total_Students  
FROM Student  
GROUP BY DeptID  
ORDER BY Total_Students DESC;
```

(c) Student name with Department name and HoD

```
SELECT S.Name, D.Dept_Name, D.HoD
```

```
FROM Student S, Department D  
WHERE S.DeptID = D.DeptID;
```

4. REGISTRATION FORM

Create table

```
CREATE TABLE StudentRegistration (  
  Regno INT PRIMARY KEY,  
  Name VARCHAR(50),  
  Gender VARCHAR(10),  
  DeptID INT,  
  PhoneNo VARCHAR(15)  
);
```

Insert

```
INSERT INTO StudentRegistration VALUES  
(101,'Arun','M',1,'9876543210'),  
(102,'Bala','M',1,'9123456780');
```

Display registered students

```
SELECT * FROM StudentRegistration;
```


12) 1. TABLES

```
CREATE TABLE stu_details (  
  reg_no INT PRIMARY KEY,  
  stu_name VARCHAR(50),  
  DOB DATE,  
  address VARCHAR(100),  
  city VARCHAR(50)  
);
```

```
CREATE TABLE mark_details (  
  reg_no INT,  
  mark1 INT,  
  mark2 INT,  
  mark3 INT,  
  total INT,  
  FOREIGN KEY (reg_no) REFERENCES stu_details(reg_no)  
);
```

2. INSERT

```
INSERT INTO stu_details VALUES  
(1,'Arun','2004-05-10','Chennai','Chennai'),  
(2,'Bala','2003-08-15','Madurai','Madurai');
```

```
INSERT INTO mark_details VALUES  
(1,80,85,90,255),  
(2,70,75,80,225);
```

3. QUERIES

(a) Add average column in mark_details

```
ALTER TABLE mark_details  
ADD average DOUBLE;
```

(b) Months between DOB and today

```
SELECT reg_no, stu_name,  
TIMESTAMPDIFF(MONTH, DOB, CURDATE()) AS Months  
FROM stu_details;
```

(c) Drop address column

```
ALTER TABLE stu_details  
DROP COLUMN address;
```

(d) PROCEDURE for sum & average marks

DELIMITER \$\$

```
CREATE PROCEDURE CalcMarks()  
BEGIN  
  UPDATE mark_details  
  SET total = mark1 + mark2 + mark3,  
      average = (mark1 + mark2 + mark3) / 3;  
END $$
```

DELIMITER ;

Call Procedure:

```
CALL CalcMarks();
```

13) 1. TABLES

```
CREATE TABLE Department (  
  Deptno INT PRIMARY KEY,  
  Dname VARCHAR(50),  
  Dloc VARCHAR(50)  
);
```

```
CREATE TABLE Employee (  
  Empno INT PRIMARY KEY,  
  Ename VARCHAR(50),  
  Job VARCHAR(50),  
  MgrId INT,  
  DoB DATE,  
  DoJ DATE,  
  Sal DECIMAL(10,2),  
  Comm DECIMAL(10,2),  
  Deptno INT,  
  FOREIGN KEY (Deptno) REFERENCES Department(Deptno)  
);
```

2. INSERT

```
INSERT INTO Department VALUES  
(10,'Marketing','Chennai'),  
(20,'HR','Mumbai'),  
(30,'IT','Bangalore');
```

```
INSERT INTO Employee VALUES  
(1,'Arun','Manager',NULL,'1995-05-10','2020-06-01',50000,5000,10),  
(2,'Bala','Analyst',1,'1998-07-15','2021-08-10',40000,3000,20),  
(3,'Charan','Developer',1,'1997-05-20','2022-01-05',45000,4000,10);
```

3. QUERIES

(a) Emp no, name, salary and experience ordered by salary (desc)

```
SELECT Empno, Ename, Sal,  
TIMESTAMPDIFF(YEAR, DoJ, CURDATE()) AS Experience  
FROM Employee  
ORDER BY Sal DESC;
```

(b) Employees working in Marketing department

```
SELECT E.Ename  
FROM Employee E, Department D  
WHERE E.Deptno = D.Deptno  
AND D.Dname = 'Marketing';
```

(c) Employees born in current month

```
SELECT Ename  
FROM Employee  
WHERE MONTH(DoB) = MONTH(CURDATE());
```

4. EMPLOYEE REGISTRATION FORM

Create table (form storage)

```
CREATE TABLE EmployeeForm (  
  Empno INT PRIMARY KEY,  
  Ename VARCHAR(50),  
  Job VARCHAR(50),  
  DoB DATE,  
  DoJ DATE,  
  Sal DECIMAL(10,2),  
  Deptno INT  
);
```

Insert (form submission)

```
INSERT INTO EmployeeForm VALUES  
(1,'Arun','Manager','1995-05-10','2020-06-01',50000,10),  
(2,'Bala','Analyst','1998-07-15','2021-08-10',40000,20);
```

Display employees

```
SELECT * FROM EmployeeForm;
```

14) 1. TABLES

```
CREATE TABLE Course (  
  Ccode INT PRIMARY KEY,  
  Course_name VARCHAR(50)  
);
```

```
CREATE TABLE Student (  
  Rollno INT PRIMARY KEY,  
  Name VARCHAR(50),  
  Coursecode INT,  
  Mark1 INT,  
  Mark2 INT,  
  FOREIGN KEY (Coursecode) REFERENCES Course(Ccode)  
);
```

2. INSERT

```
INSERT INTO Course VALUES  
(1,'CSE'),  
(2,'IT'),  
(3,'ECE');
```

```
INSERT INTO Student (Rollno, Name, Coursecode, Mark1, Mark2) VALUES  
(101,'Arun',1,80,85),  
(102,'Bala',2,70,75),  
(103,'Charan',1,90,88),  
(104,'Divya',3,85,80),  
(105,'Esha',2,78,82);
```

3. QUERIES

(a) Add TOTAL field to Student

```
ALTER TABLE Student  
ADD Total INT;
```

(b) Insert 5 records (already done above)

```
INSERT INTO Student (Rollno, Name, Coursecode, Mark1, Mark2,Total) VALUES  
(101,'Arun',1,80,85,165),  
(102,'Bala',2,70,75,145),  
(103,'Charan',1,90,88,178),  
(104,'Divya',3,85,80,165),  
(105,'Esha',2,78,82,160);
```

(c) Student names with course names

```
SELECT S.Name, C.Course_name
FROM Student S, Course C
WHERE S.Coursecode = C.Ccode;
```

4. PL/SQL / MYSQL PROCEDURE

```
DELIMITER $$
```

```
CREATE PROCEDURE OddEven(IN num INT)
BEGIN
  IF num % 2 = 0 THEN
    SELECT 'EVEN NUMBER' AS Result;
  ELSE
    SELECT 'ODD NUMBER' AS Result;
  END IF;
END $$
```

```
DELIMITER ;
```

Call Procedure

```
CALL OddEven(10);
```

15) 1. TABLES

```
CREATE TABLE Department (  
  Depno INT PRIMARY KEY,  
  Depname VARCHAR(50),  
  Deplocation VARCHAR(50)  
);
```

```
CREATE TABLE Employee (  
  Empno INT PRIMARY KEY,  
  Empname VARCHAR(50),  
  DOJ DATE,  
  Salary DECIMAL(10,2),  
  Depno INT,  
  FOREIGN KEY (Depno) REFERENCES Department(Depno)  
);
```

2. INSERT

```
INSERT INTO Department VALUES  
(10,'HR','Chennai'),  
(20,'IT','Bangalore'),  
(30,'Sales','Mumbai');
```

```
INSERT INTO Employee VALUES  
(1,'Arun','2020-06-01',50000,10),  
(2,'Bala','2021-08-10',40000,20),  
(3,'Charan','2019-01-15',60000,10),  
(4,'Divya','2022-05-20',45000,20);
```

3. QUERIES

(a) Number of employees in each department (deptno desc)

```
SELECT Depno, COUNT(*) AS Total_Employees  
FROM Employee  
GROUP BY Depno  
ORDER BY Depno DESC;
```

(b) Departments having at least one employee

```
SELECT DISTINCT D.Depname  
FROM Department D, Employee E  
WHERE D.Depno = E.Depno;
```

(c) Employee name, DOJ and experience (years)

```
SELECT Empname, DOJ,
```

```
TIMESTAMPDIFF(YEAR, DOJ, CURDATE()) AS Years_Completed  
FROM Employee;
```

4. EMPLOYEE FORM

Create table

```
CREATE TABLE EmployeeForm (  
  Empno INT PRIMARY KEY,  
  Empname VARCHAR(50),  
  DOJ DATE,  
  Salary DECIMAL(10,2),  
  Depno INT  
);
```

Insert

```
INSERT INTO EmployeeForm VALUES  
(1,'Arun','2020-06-01',50000,10),  
(2,'Bala','2021-08-10',40000,20);
```

Display employee details

```
SELECT * FROM EmployeeForm;
```

Delete employee record

```
DELETE FROM EmployeeForm  
WHERE Empno = 2;
```


16) 1. TABLES

```
CREATE TABLE order_master (  
  Orderno INT PRIMARY KEY,  
  Vencode VARCHAR(10),  
  Ordstatus VARCHAR(20),  
  Delv_date DATE  
);
```

```
CREATE TABLE order_detail (  
  Orderno INT,  
  Itemcode INT,  
  Qty_ord INT,  
  Qty_deld INT,  
  FOREIGN KEY (Orderno) REFERENCES order_master(Orderno)  
);
```

2. INSERT

```
INSERT INTO order_master VALUES  
(101,'V001','Delivered','2006-02-10'),  
(102,'V004','Pending','2006-03-15'),  
(103,'V004','Pending','2006-06-20'),  
(104,'V002','Delivered','2006-07-10');
```

```
INSERT INTO order_detail VALUES  
(101,1,10,10),  
(102,2,20,10),  
(103,3,15,0),  
(104,4,25,25);
```

3. QUERIES

(a) Orders handled by vendor V004 ordered by delivery date

```
SELECT *  
FROM order_master  
WHERE Vencode = 'V004'  
ORDER BY Delv_date;
```

(b) Orders placed between 01-Jan-2006 and 01-Jul-2006

```
SELECT *  
FROM order_master  
WHERE Delv_date BETWEEN '2006-01-01' AND '2006-07-01';
```

(c) Pending orders

```
SELECT *  
FROM order_master  
WHERE Ordstatus = 'Pending';
```

4. PROCEDURE (Leap Year Check – MySQL)

```
DELIMITER $$
```

```
CREATE PROCEDURE CheckLeapYear(IN yr INT)  
BEGIN  
    IF (yr % 400 = 0) OR (yr % 4 = 0 AND yr % 100 != 0) THEN  
        SELECT 'LEAP YEAR' AS Result;  
    ELSE  
        SELECT 'NOT LEAP YEAR' AS Result;  
    END IF;  
END $$
```

```
DELIMITER ;
```

Call Procedure

```
CALL CheckLeapYear(2024);
```